



# CMPT 295

Unit – Memory Hierarchy

Lecture 38 – Cache Memories

# Last Lecture

- Over the years, the performance of microprocessors has been increasing faster than memory performance (time it takes to access memory) creating a growing **gap** between them
  - This gap represents the large time difference between **accessing registers** and **accessing main memory**
- To narrow this gap, a **cache** is added to memory forming a **memory hierarchy**
  - **Cache** uses faster memory components (SRAM: 4 clock cycles per access) to hold copy of data/instructions held in main memory, data/instructions which is likely to be used in near future, i.e., in the following memory accesses
  - **Cache** takes advantage of **locality**
    - **Temporal locality**: access the same memory location again soon
    - **Spatial locality**: access a nearby memory location soon
- Understanding **locality**, **memory hierarchy** and **caching** allows us to write **cache-friendly code**
  - Programs with **good locality** tend to run faster because such code maximizes the use of data/instructions found in cache by accessing same memory location again and again (**temporal locality**) and/or by accessing nearby locations held in cache (**spatial locality**)

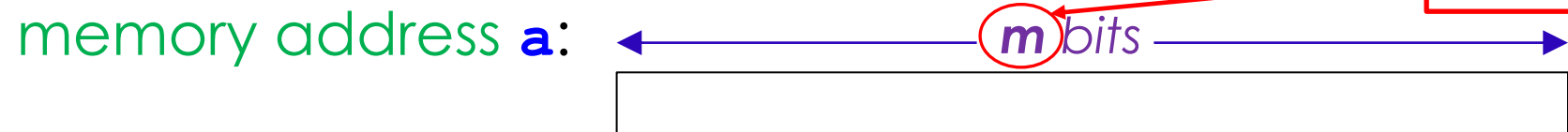
# Today's Menu

- Cache-friendly code optimization:
  - Locality
  - Memory hierarchy
  - Caching
- Cache memories
  - How does a cache work?
  - How is cache structured?
  - Examples
  - Cache performance analysis
  - Another example of cache -> conflict miss
- Memory mountain

# How does **cache** work? In details!

From our last lecture:

- ▶ When microprocessor executes a **LOAD** instruction
  - ▶ **LOAD** *a*, *rC* -> read data from memory at memory address *a*
  - ▶ Microprocessor actually reads the content of memory address *a* from **L1 cache** layer of the **memory hierarchy** instead of from **main memory** layer of the **memory hierarchy**



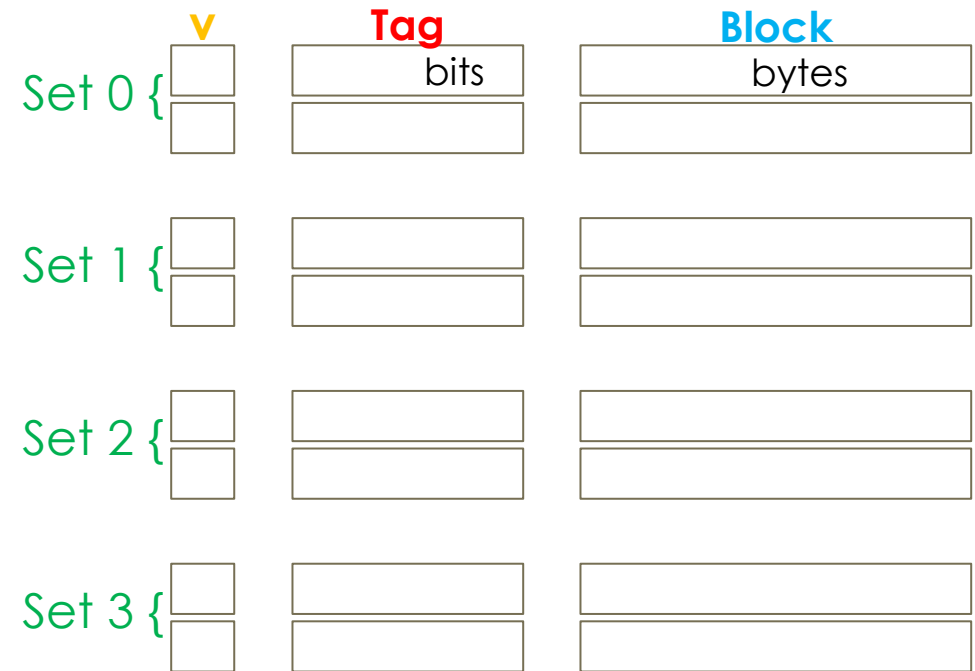
From our lectures on ISA, size of memory:  
 $2^m \times n$  bits

- ▶ How does the **cache** know it contains a valid copy of content of memory address *a*?

# How is **cache** structured?

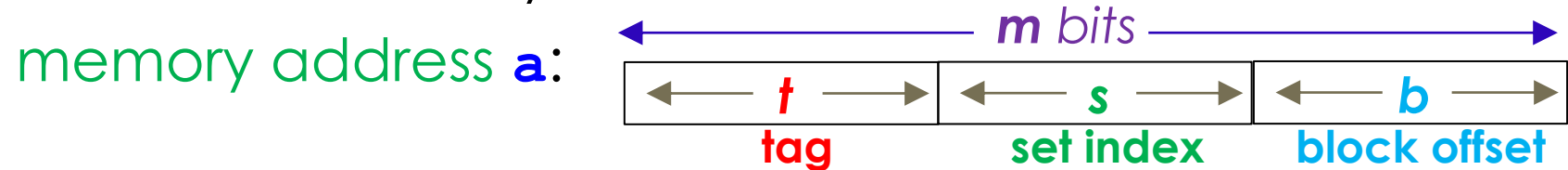
## L1 Cache

- Memory model (from ISA):
  - Size of memory:  $M = 2^m \times n$  bits
  - Byte-addressable memory ( $n = 8$  bits) (memory resolution)
  - Cache **C** contains **S** sets =  $2^s$  sets
    - Each set contains **E** cache lines
    - Each line contains:
      - 1 **data** block **B** =  $2^b$  bytes
      - 1 valid bit **v**
      - **t** tag bits, where  $t = m - (b + s)$
    - Defined as tuple (**S**, **E**, **B**, **m**)
    - Capacity (size) of **C** = **S** x **E** x **B** bytes



# Back to our question

- How does the **cache** know it contains a valid copy of content of memory address **a** ?

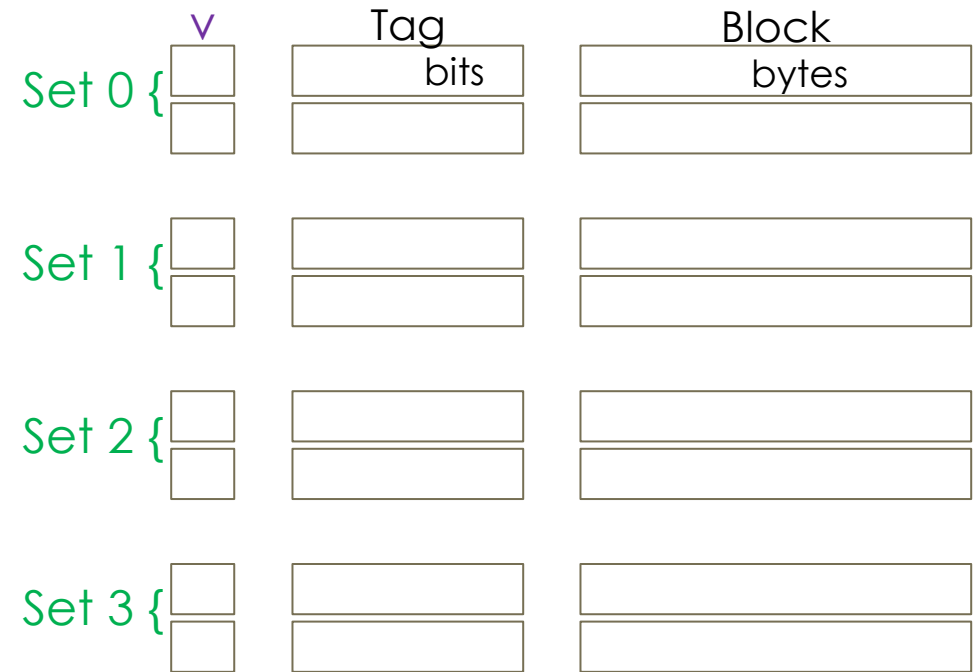


- s** -> number of bits uniquely identifying a **set** (**set index**)
  - Set index** tells us in which **set** in the cache to look for requested data
- t** -> number of bits defining the **tag**
  - Tag** tells us in which active line of a **set** in the cache to look for requested data
- b** -> number of bits defining the **block offset**
  - Block offset** tells us where to find requested data within **block B** in the cache

Read as  
unsigned  
integers

Example: Capacity of **cache** defined as  $(S, E, B, m) = (4, 2, 64, 64)$  ?

- Memory model (from ISA):
  - Size of memory:  $\mathbf{M} = 2^m \times n$  bits
  - Byte-addressable memory ( $n = 8$ ) (memory resolution)
  - Cache  $\mathbf{C}$  contains  $\mathbf{S}$  sets =  $2^s$  sets
    - Each set contains  $\mathbf{E}$  cache lines
    - Each line contains:
      - 1 data block  $\mathbf{B} = 2^b$  bytes
      - 1 valid bit  $\mathbf{v}$
      - $\mathbf{t}$  tag bits, where  $\mathbf{t} = m - (b + s)$
  - Defined as tuple  $(\mathbf{S}, \mathbf{E}, \mathbf{B}, m)$
  - Capacity (size) of  $\mathbf{C} = \mathbf{S} \times \mathbf{E} \times \mathbf{B}$  bytes



$S =$   $\rightarrow s =$   
 $E =$   
 $B =$   $\rightarrow b =$   
 $m =$   $t = m - (b + s) =$   
 Capacity of  $C =$

# Example: L1 cache $\rightarrow (S, E, B, m) = (4, 2, 64, 64)$

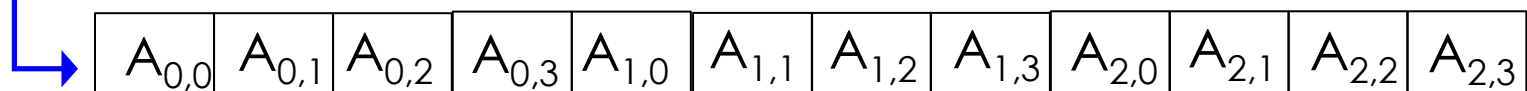
Assuming memory hierarchy: CPU registers – L1 cache – main memory

- Let's investigate how L1 cache is used when `sumArray1(...)` executes

```
// M = 3, N = 4
int sumArray1(int A[M][N]) {
    int i, j, sum;
    sum = 0;
    for (i = 0; i < M; i++) {
        for (j = 0; j < N; j++)
            sum += A[i][j];
    }
    return sum;
}
```

1. Draw the array in memory and its associated memory addresses:

Memory address: `0x00007FF6FF5EFD80` ...D84 ...D88 ...D8C ...D90 ...D94 ...D98 ...D9C ...DA0 ...DA4 ...DA8 ...DAC





# Example: L1 cache $\rightarrow (S, E, B, m) = (4, 2, 64, 64)$

Assuming memory hierarchy: CPU registers – L1 cache – main memory

1. Iteration 1 of `sumArray1()`:

C: `sum += A[0][0];`

Assembly instruction: `addl (%rdi,%rcx,4),%eax`

One of the micro operations of `addl`: `load 00007FF6FF5EFD80`

➡ CPU sends a `load` request to L1 cache

0. Memory address expansion (into bit pattern):

`00007FF6FF5EFD80`:   
Tag  $t = 00007FF6FF5EFD$       Block offset = 0

1. Set Selection:

2. Line Matching: Is  $v$  valid?

Inactive **set** line  $\rightarrow v = 0$  ( $v$  invalid)

Active **set** line  $\rightarrow v = 1$  ( $v$  valid)

|         | v | Tag | Block |
|---------|---|-----|-------|
| Set 0 { | 0 |     |       |
|         | 0 |     |       |
| Set 1 { | 0 |     |       |
|         | 0 |     |       |
| Set 2 { | 0 |     |       |
|         | 0 |     |       |
| Set 3 { | 0 |     |       |
|         | 0 |     |       |

L1 cache is empty (since we have just started executing `sumArray1()`).

This is called a **cold cache** and this type of **cache miss** is called **cold miss**.

# Example: L1 cache $\rightarrow (S, E, B, m) = (4, 2, 64, 64)$

Assuming memory hierarchy: CPU registers – L1 cache – main memory

## 1. Iteration 1 of sumArray1 () (cont'd):

00007FF6FF5EFD80:  $\xleftarrow{56}$  0...011111110101111011111101  $\xrightarrow{-2}$  10  $\xleftarrow{6}$  000000

Tag  $t = 00007FF6FF5EFD$  Block offset = 0

- So now, the content of 00007FF6FF5EFD80 is read from main memory and copied into Set 2 of L1 cache

- **Actually**, it isn't just the content of 00007FF6FF5EFD80 that is read from main memory but 64 bytes starting at 00007FF6FF5EFD80, i.e., from 00007FF6FF5EFD80 to 00007FF6FF5EFD8F (inclusively) is read from main memory and copied into the block of Set 2 of L1 cache  $\rightarrow$  expensive (takes time)

- Tag  $t$  set to 00007FF6FF5EFD and  $v$  set 1 (i.e. valid)

- **Word extraction:** L1 cache sends content of block at block offset 0 to microprocessor  $\rightarrow$  4 bytes (int)

|         | V                         | Tag                                   | Block                                                        |
|---------|---------------------------|---------------------------------------|--------------------------------------------------------------|
| Set 0 { | <div>0</div> <div>0</div> | <div></div> <div></div>               | <div></div> <div></div>                                      |
| Set 1 { | <div>0</div> <div>0</div> | <div></div> <div></div>               | <div></div> <div></div>                                      |
| Set 2 { | <div>1</div> <div>0</div> | <div>00007FF6FF5EFD</div> <div></div> | <div>0 1 2 3 ... 63</div> <div>A[0][0] ...</div> <div></div> |
| Set 3 { | <div>0</div> <div>0</div> | <div></div> <div></div>               | <div></div> <div></div>                                      |

# Example: L1 cache $\rightarrow (S, E, B, m) = (4, 2, 64, 64)$

Assuming memory hierarchy: CPU registers – L1 cache – main memory

## 2. Iteration 6 of `sumArray1()`:

C: `sum += A[1][1];`

Assembly instruction: `addl (%rdi,%rcx,4),%eax`

One of the micro operations of `addl: load 00007FF6FF5EFD94`

➔ CPU sends a `load` request to L1 cache

### 0. Memory address expansion (into bit pattern):

`00007FF6FF5EFD94`:   
Tag  $t = 00007FF6FF5EFD$       Block offset = 20

### 1. Set Selection: $10_2 \rightarrow 2_{10}$

### 2. Line Matching: Is $v$ valid? 1 $\rightarrow$ valid!

➔ Is tag matching? Yes, then **L1 cache hit**

### 3. Word extraction: L1 cache sends content of **block** at **block offset 20** to microprocessor $\rightarrow$ 4 bytes (int)

|         | V                         | Tag                                   | Block                                                                    |
|---------|---------------------------|---------------------------------------|--------------------------------------------------------------------------|
| Set 0 { | <div>0</div> <div>0</div> | <div></div> <div></div>               | <div></div> <div></div>                                                  |
| Set 1 { | <div>0</div> <div>0</div> | <div></div> <div></div>               | <div></div> <div></div>                                                  |
| Set 2 { | <div>1</div> <div>0</div> | <div>00007FF6FF5EFD</div> <div></div> | <div>0 ... 20 ... 63</div> <div>... <b>A[1][1]</b> ...</div> <div></div> |
| Set 3 { | <div>0</div> <div>0</div> | <div></div> <div></div>               | <div></div> <div></div>                                                  |

$A_{00} = A[0][0] = 00007FF6FF5EFD80$   
 $A_{01} = A[0][1] = 00007FF6FF5EFD84$   
 $A_{02} = A[0][2] = 00007FF6FF5EFD88$   
 $A_{03} = A[0][3] = 00007FF6FF5EFD8C$   
 $A_{10} = A[1][0] = 00007FF6FF5EFD90$   
 $A_{11} = A[1][1] = 00007FF6FF5EFD94$

# Performance Analysis of `sumArray1`

- Total number of elements accessed in memory:
- Cache miss:
- Miss rate:
- Cache hit:
- Hit rate:
- What is the average memory access time?

average memory access time = hit rate  $\times$  hit time + miss rate  $\times$  miss time

# Types of cache - ( $S, E, B, m$ )

1. If  $E = 1 \rightarrow$  one line per *set*
  - *Direct-mapped cache*
  - Simple “line matching” process
  - Issue: conflict miss
2. If  $S = 1 \rightarrow$  one *set*
  - *Fully-associative cache*
  - No *set* index bits in memory address
  - Issue: *Tag* comparisons – done in parallel
    - Expensive to build
3. Our example used a *set associative cache*

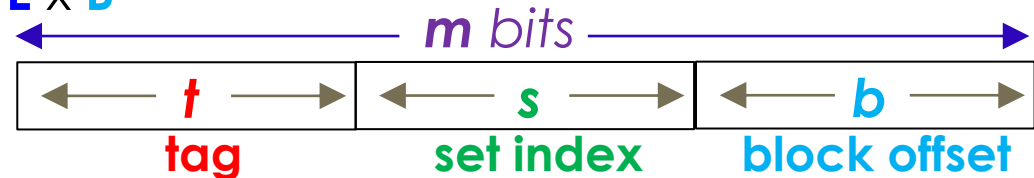
# Summary

- A **cache C** defined as (**S**, **E**, **B**, **m**) with memory of size of  $2^m$  has:
  - **S** cache **sets**
  - Each **set** contains **E** cache lines
  - Each line contains:
    - 1 data block **B** of  $2^b$  bytes, 1 valid bit **v**, **t** tag bits, where  $t = m - (b + s)$

- Capacity (size) of **C** = **S** x **E** x **B**

- Memory address:

- To find data in **L1 cache**:



0. Memory address expansion (into bit pattern)

1. Set Selection: Select cache **set** using **set index**

2. Line Matching: If **v** valid, search **set** for matching **tag**

-> if found: **cache hit**, -> if not found: **cache miss**

- If **cache miss** occurs -> must search memory hierarchy at next lower level (expensive)

3. Word extraction: When **cache hit**, **L1 cache** sends content of **block** at **block offset** to microprocessor

# Next Lecture

- Cache-friendly code optimization:
  - Locality
  - Memory hierarchy
  - Caching
- Cache memories
  - How does a cache work?
  - How is cache structured?
  - Examples
  - Cache performance analysis
  - Another example of cache -> conflict miss
- Memory mountain